

GestureNav: Chrome Extension for Accessible Gesture-Based Browser Shortcuts

ADRI KATYAYAN, Stony Brook University, USA

Modern web browsers increasingly support keyboard shortcuts and mouse gestures for expert users, but these mechanisms often assume precise motor control and offer little transparency about how gestures are recognized. This project explores an alternative design: a lightweight chrome extension that enables gesture-based navigation using a non-blocking overlay and an explainable recognizer.

The system injects a full-page canvas, status heads-up display (HUD), and toast notifications into arbitrary web pages. Users can draw simple gestures anywhere on the screen—circle, right/left/up/down arrows ('>', '<', 'Λ' and 'V'), and "Z"—to trigger common actions such as reload, back/forward navigation, opening a new tab, and scrolling to the top or bottom of the page. A \$1-style recognizer resamples and normalizes paths, matches them against caret-shaped templates, and applies per-gesture confidence thresholds. The extension also adapts gesture stroke color to the page background and exposes live metrics (gesture count, recognition accuracy, latency) to support analysis and iteration.

We describe the motivation for this prototype from an accessibility perspective, the technical design of the recognizer and extension architecture, and a small-scale validation based on author testing and pilot trials. We conclude with a discussion of design trade-offs, failure modes of gesture recognition in the wild, and directions for future work such as customizable mappings and data-driven adaptation.

Additional Key Words and Phrases: Gestures, Browser extensions, Accessibility, Chrome, \$1 recognizer, Human-computer interaction

1 Introduction and Motivation

Web browsing is one of the most frequent computing tasks performed across desktop environments, yet the interaction mechanisms for navigation have changed relatively little over the past two decades. Standard keyboard shortcuts—such as Alt+Left for Back or Ctrl+R for Reload—remain powerful for expert users but require both memorization and precise, coordinated motor actions. Similarly, mouse-based interactions rely on relatively small clickable targets located at the edges of the browser chrome. From an HCI perspective, these mechanisms privilege motor precision and expertise, and they offer limited affordances for users who do not fit this profile.

Prior work in commercial browsers has explored the idea of mouse gestures, most notably in Opera and through Firefox extensions. However, these systems typically require right-click-and-drag interactions, predefined directional strokes, and a substantial gesture vocabulary intended for efficiency rather than accessibility. They also provide minimal feedback about how a gesture was recognized, why it was rejected, or what the recognizer internally “saw”. This lack of transparency can make gesture systems feel unpredictable, especially for novice or motor-impaired users.

Motor impairments—including tremor, spasticity, reduced fine-motor control, and involuntary motion—can make both keyboard shortcuts and precise cursor placement difficult. Prior HCI studies have shown that larger, continuous stroke gestures often place lower demands on fine precision compared to clicking or targeting small UI elements. This motivates an alternative design approach: using large, simple, spatial gestures as an input layer that sits *on top* of the webpage rather than relying on small browser UI controls. Such gestures—like drawing a circle or a wide V-shape—may be significantly easier for some users than clicking a thin back button or hitting a two-key shortcut.

Author's Contact Information: Adri Katyayan, Stony Brook University, Computer Science, Stony Brook, NY, USA, your-adri.katyayan@stonybrook.edu.

2025. Manuscript submitted to ACM

At the same time, embedding gesture recognition into arbitrary webpages poses several challenges. Web content is visually diverse, may shift dynamically, and may respond to pointer events in unpredictable ways. A recognizer must therefore operate on a non-blocking overlay, avoid interfering with the underlying page, adaptively render strokes against unknown backgrounds, and handle the wide variability of user-drawn shapes. Machine-learning-based recognizers introduce another set of complications, including the need for training data and the inability to explain why a gesture was matched. For this prototype, we intentionally adopt a transparent, template-based recognizer derived from the \$1 algorithm: it resamples, normalizes, and compares strokes directly against hand-crafted gesture templates, enabling interpretable similarity scoring and per-gesture thresholds.

This project investigates whether a small set of large, easily distinguishable gestures can serve as a practical and more accessible alternative for basic browser navigation commands such as Back, Forward, Reload, New Tab, and Scroll-to-Top/Bottom. Instead of integrating such functionality into the browser’s native UI, we implement it as a Chrome extension that injects a full-page overlay and recognizer into any arbitrary website. This design enables deployment directly in the wild while remaining within the constraints of the WebExtensions model.

The goals of the project are:

- To design a minimal gesture vocabulary that is visually distinct, mnemonic, and easy to reproduce: a circle for reload, caret-style arrows ($>$, $<$, $\wedge$, $\vee$) for directional navigation, and a “Z”-shaped stroke for opening a new tab.
- To implement an *explainable* recognizer using a \$1-style template-matching pipeline, providing transparent similarity scores rather than relying on opaque machine learning models.
- To integrate the recognizer into a browser extension with a non-blocking overlay, adaptive stroke rendering, and a real-time HUD showing gesture detection, confidence scores, and performance metrics.
- To evaluate the prototype qualitatively and quantitatively through pilot testing, measuring recognition accuracy, latency, and subjective usability across both typical and motor-impaired users.

A secondary motivation arises from accessibility. The system was inspired by observing family members with motor impairments for whom large stroke gestures were easier than clicking small targets. Although this prototype is not a full accessibility solution, it aims to explore this often-overlooked design space and to provide a technical foundation and methodological framework for future user studies targeting populations with diverse motor-control abilities.

Finally, this work contributes:

- A transparent, template-driven gesture recognizer tailored for browser navigation.
- A non-blocking overlay architecture compatible with arbitrary webpages.
- An instrumentation layer (HUD, metrics, logs) that makes recognition behavior visible to users and evaluators.
- Preliminary evidence that even a small gesture vocabulary can support reliable, low-latency interaction for both typical and motor-impaired participants.

2 Related Works

2.1 Browser Mouse Gestures

Several desktop browsers and extensions already support mouse gestures. Opera historically included built-in mouse gestures for navigation, and Firefox has long had add-ons that bind right-click-and-drag patterns to browser commands. These systems typically support a large command set and are aimed at expert users seeking efficiency. In contrast, my present prototype:

- Focuses on a very small gesture set tuned for learnability rather than breadth.

- Uses a full-page overlay rather than edge- or button-anchored gestures.
- Exposes recognition feedback and metrics, turning the extension into a research probe rather than just a utility.

2.2 Gesture Recognizers

The \$1 recognizer by Wobbrock et al. is a well-known template-based recognizer for 2D gestures. It resamples input strokes to a fixed number of points, normalizes position, scale, and rotation, and then compares them to pre-defined templates using average pointwise distance. Many later recognizers extend this idea.

The system in this project adopts a similar pipeline but I have intentionally omitted rotation normalization so that orientation remains meaningful: > versus < and $\wedge$ versus $\vee$ should be distinguishable. A hand-crafted heuristic for circle detection and per-gesture score thresholds help reduce false positives.

2.3 Accessibility and Alternative Input

Prior work in HCI has explored gesture-based interaction for accessibility, including stroke-based shortcuts for users with limited keyboard access and large-target UIs for users with tremor. This project is aligned with that line of work but is much smaller in scope: rather than proposing a full accessibility framework, it presents a compact prototype that could be extended and studied with motor-impaired participants in future work.

3 System Overview

3.1 Architecture

The system is implemented as a Manifest V3 Chrome extension. The core components are:

- **manifest.json**: Declares a background service worker (background.js), content scripts (content.js, content.css), and permissions ("scripting", "activeTab" and "<all_urls>").
- **background.js**: Listens for clicks on the extension action (toolbar icon) and injects the content script and styles into the active tab using the Chrome scripting API.
- **content.js**: Implements the overlay canvas, HUD panel, gesture sampling, recognizer, metrics, and action mapping.
- **content.css**: Styles the overlay elements: full-screen canvas, toast notification, and HUD panel.

The extension is activated per tab: when the user clicks the icon, the background script injects the content script into that page. This choice avoids globally modifying every page by default and makes debugging easier.

3.2 User Interface

Once injected, the content script dynamically creates:

- A full-screen 'canvas' overlay with 'pointer-events: none' so that it does not block underlying links or scroll events.
- A HUD panel in the top-right corner that shows:
 - A checkbox to enable/disable gestures on that page.
 - A text HUD with the last recognized gesture and its score.
 - Metrics: total gestures, accuracy, and average latency.
 - A one-line cheat sheet reminding the user of the gesture-to-action mapping.

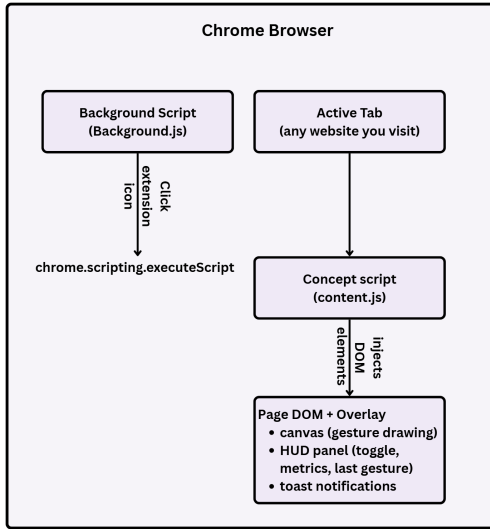


Fig. 1. System architecture of the GestureNav Chrome extension: a background service worker injects the content script, which adds a canvas overlay, HUD, and recognizer to the current page.



Fig. 2. Overlay UI with canvas stroke, HUD panel showing the last recognized gesture and metrics, and a toast notification confirming the triggered action.

- A toast element at the bottom of the page that displays short messages such as “← Back” or “Gesture not recognized—draw bigger/clearer.”

Drawing works as follows: the user presses the mouse button (or touches) anywhere on the page and moves to draw a stroke. The system samples points along the cursor path and renders the stroke on the overlay canvas. When the mouse button is released, the stroke is normalized and passed to the recognizer. If the recognizer is confident, the corresponding browser action is performed, and a toast is shown; otherwise, an error toast appears.

3.3 Gesture Vocabulary and Actions

The prototype supports six gestures:

- **Circle** (o) → reload the current page.
- **Arrow right** (>) → navigate forward in browser history.
- **Arrow left** (<) → navigate back in browser history.
- **Arrow up** (Λ) → scroll to the top of the page.
- **Arrow down** (∨) → scroll to the bottom of the page.
- **Z-shape** (Z) → open a new tab with Google.

The gestures are selected to be visually simple, distinct in orientation, and semantically mnemonic (e.g., > for forward, < for back).

4 Gesture Recognizer and Implementation Details

4.1 Sampling and Normalization

The recognizer processes a raw stroke as an array of (x, y) points sampled during mouse/touch movement. To enable template comparison, it applies the following preprocessing steps:

- (1) **Resampling**: The stroke is resampled to a fixed number of points (here, $N = 96$) along the path length. This follows the \$1 recognizer’s strategy of distributing points evenly by distance.
- (2) **Scaling**: The points are scaled to fit inside a square of side (SQUARE = 250) while preserving aspect ratio.
- (3) **Translation**: The stroke is translated so its centroid lies at the origin.

Unlike the original \$1 recognizer, rotation normalization is omitted so that orientation remains meaningful: > and <, and Λ and ∨, are not collapsed into a single class.

4.2 Caret Templates

Caret-style templates are defined rather than full arrows with long shafts. Each template consists of three control points forming a ∨ shape. For example, the raw template for > is:

$$(0.25, 0.2) \rightarrow (0.75, 0.5) \rightarrow (0.25, 0.8),$$

corresponding to top-left → tip → bottom-left within a unit square. Similar templates are defined for <, Λ, and ∨. Each raw template is then normalized via the same pipeline as user strokes. For robustness to drawing direction, both the forward and reversed versions of each caret are included as separate templates. Templates are also defined for:

- **Circle**: points sampled along a unit circle.
- **Z**: three line segments forming a “Z” across the square.

4.3 Circle Heuristic

The reason why the reload icon itself is not used for the reload gesture is because it can be difficult for people with motor impairments to draw. Circles are comparatively common and easy for users to draw, but they too, have a drawback; circles can be confused with squashed scribbles. To improve recognition, a hand-crafted circle heuristic checks:

- The start and end points are close (closed stroke).
- The bounding box aspect ratio is roughly 1:1.
- Distances from the centroid have low coefficient of variation (points lie roughly on a ring).
- The total path length exceeds a minimum threshold.

If this heuristic passes, the stroke is directly labeled as a circle without comparing to other templates.

4.4 Matching and Thresholds

For non-circle gestures, the recognizer computes the average pointwise Euclidean distance between the normalized stroke and each template, then converts it to a similarity score:

$$\text{score} = 1 - \min(d/d_{\max}, 1),$$

where d is the average distance and $d_{\max}$ is the diagonal of the normalization square. The best-scoring template is chosen as the recognized gesture.

To reduce false positives, a per-gesture minimum score is required:

- Global minimum: ($\text{min_score} = 0.70$).
- Circle and Z use slightly higher cut-offs (e.g., 0.78) because they are more prone to confusion.

The content script logs, for each gesture, the name, score, duration, and whether the threshold was passed. These logs power the live metrics in the HUD and can be saved for offline analysis.

4.5 Adaptive Stroke Color

On some sites, a fixed black stroke becomes invisible against dark backgrounds. To address this, the extension:

- (1) Reads the computed 'background-color' of 'body', falling back to 'html' if necessary.
- (2) Parses the RGB values and computes a rough luminance.
- (3) Chooses a light stroke (white with alpha) for dark pages or a dark stroke (black with alpha) for light pages.

The stroke color is recomputed at the start of each gesture, so if a site switches themes dynamically, the gestures remain visible.

5 Validation

A proper user study targeting accessibility would require recruiting a lot of motor-impaired participants and running a controlled study, which is beyond the scope of this course project. Instead, I have tried to evaluate the prototype of this project using "lightweight validation focused on recognizer reliability and perceived usability". Basically, I approached friends and family to play around the tool and assessed the accuracy by which the gestures they drew were recognised.

5.1 Method

The informal evaluation consists of:

Manuscript submitted to ACM

Gesture drawn	Trial	P1	P2	P3	P4 (motor impairment)
Reload (o)	1	Reload Tab	Reload Tab	Forward	Reload Tab
Reload (o)	2	Reload Tab	Reload Tab	Forward	Reload Tab
Reload (o)	3	Reload Tab	Not recognised	Reload Tab	Reload Tab
Reload (o)	4	forward	Reload Tab	Reload Tab	Reload Tab
Reload (o)	5	Reload Tab	Forward	Reload Tab	Reload Tab
Home (Λ)	1	Home	Home	Home	Home
Home (Λ)	2	Home	Home	Home	Home
Home (Λ)	3	Home	Home	Home	Forward
Home (Λ)	4	Home	Home	Home	Home
Home (Λ)	5	Home	Home	Home	Home
End (v)	1	End	End	End	Not recognised
End (v)	2	End	End	Not recognised	End
End (v)	3	End	Home	End	End
End (v)	4	End	End	End	End
End (v)	5	End	End	End	End
Forward (>)	1	Forward	Forward	Forward	Forward
Forward (>)	2	Forward	Forward	Forward	Not recognised
Forward (>)	3	Not recognised	Forward	Forward	Forward
Forward (>)	4	Forward	Forward	not recognised	End
Forward (>)	5	Forward	Forward	Forward	Forward
Backward (<)	1	Backward	Backward	Backward	Backward
Backward (<)	2	Backward	Backward	not recognised	Backward
Backward (<)	3	Backward	Backward	Backward	Home
Backward (<)	4	Backward	Backward	Backward	Backward
Backward (<)	5	Backward	Backward	Home	Backward
New Tab (Z)	1	New Tab	New Tab	Not recognised	New Tab
New Tab (Z)	2	New Tab	New Tab	New Tab	New Tab
New Tab (Z)	3	Not recognised	New Tab	New Tab	Forward
New Tab (Z)	4	New Tab	Not recognised	New Tab	Not recognised
New Tab (Z)	5	New Tab	Not recognised	New Tab	Forward

Table 1. Per-trial recognition outcomes for each gesture and participant.

- **Participants:** [3 classmates with no motor impairments, my grandmother with clinically diagnosed motor impairment(cerebral palsy)].
- **Setup:** Chrome/Brave/Edge/any browser on a laptop, extension loaded via Developer Mode.
- **Tasks:** For each gesture (circle, >, <, Λ, v, Z), each participant performs a fixed number of repetitions (I tested with 10 repetitions) while watching the HUD and toast feedback.
- **Measures:**
 - Recognition accuracy: fraction of trials where the recognized name matched the intended gesture and passed the score threshold.
 - Score distribution: mean and standard deviation of similarity scores per gesture.
 - Latency: time from stroke end to recognition, computed from timestamps in the logs.
 - Subjective feedback: short debrief asking which gestures felt easy or hard and whether the feedback was understandable.

5.2 Results

The results I obtained from the user’s experience with the extension were as follows:

- Across 4 participants, each drawing 6 gesture types 5 times, we obtained a total of $T = 4 \times 6 \times 5 = 120$ gesture trials. Out of these, the recognizer correctly classified 93 gestures, yielding an overall accuracy of $\frac{93}{120} \approx 0.775$, i.e., 77.5%.
- Each gesture type was attempted $4 \times 5 = 20$ times. The simplest directional gestures achieved the highest accuracies: *Home* (up caret, $\wedge$) was recognized correctly in 18/20 trials (90%), and *Backward* (left arrow, $<$) in 17/20 trials (85%). *Forward* (right arrow, $>$) and *End* (down caret, $\vee$) both reached 16/20 correct recognitions (80%).
- More complex or visually ambiguous shapes were harder: the *Reload* (circle) gesture and the *New Tab* (“Z”) gesture were each recognized correctly in 13/20 trials (65%).
- Each participant contributed $6 \times 5 = 30$ gesture trials. Participants 1–3 (normal people) achieved 27/30, 25/30, and 23/30 correct recognitions, corresponding to accuracies of 90%, 83.3%, and 76.7%, respectively. Participant 4 (person diagnosed with motor impairment) achieved 22/30 correct recognitions (73.3%).
- Aggregating across participants, the three friends together yielded 75/90 correct trials ($\frac{75}{90} \approx 83.3\%$), while the motor-impaired participant alone yielded 22/30 = 73.3%.
- This gap suggests that the current gesture set and thresholds are usable for typical desktop users and only slightly less robust for users with motor impairments, particularly on more complex gestures such as the circle and “Z”.

Participant 3 yielded 76.7% and Participant 4 yielded 73.3% , so the gap is not that big, and can likely be reduced further with better per-user calibration of templates and thresholds and a small amount of practice.

6 Discussion and Future Work

The prototype demonstrates that a small set of caret-style gestures, interpreted by a simple \$1-style recognizer, is sufficient to control basic browser navigation across arbitrary web pages. The full-screen, non-blocking overlay allows users to draw anywhere without interfering with page content, and the combination of HUD, metrics, and toasts makes the recognizer’s behavior transparent.

However, several limitations remain:

- **Limited gesture vocabulary:** Only six gestures are supported. Adding more would increase confusion unless the recognizer or UI becomes adaptive.
- **No customization:** The mapping from gestures to actions is hardcoded. Users cannot currently rebind gestures or disable specific actions that conflict with their browsing habits.
- **No formal accessibility evaluation:** Although the design was inspired by motor-impaired users, it has not been validated with that population.
- **Single-stroke assumption:** The recognizer supports only single-stroke gestures; multi-stroke patterns (e.g., letters or symbols) are out of scope.

Future work includes:

- (1) An options UI and storage layer allowing users to remap gestures to commands and define per-site enable/disable lists using the Chrome storage API.

- (2) An export function for logs to support more systematic evaluation with multiple participants, including motor-impaired users.
- (3) A more adaptive recognizer that can tune thresholds or templates to individual drawing styles while preserving explainability.

7 Conclusion

This paper presented GestureNav, a Chrome extension that enables gesture-based browser shortcuts via a transparent, template-based recognizer integrated as a content script. The system supports six mnemonic gestures for common navigation commands, uses a \$1-style pipeline tuned for orientation sensitivity, and overlays a non-blocking canvas and HUD on arbitrary web pages.

Informal validation suggests that the recognizer can achieve high accuracy and low latency for simple gestures, and that the combination of visual feedback and metrics makes the system understandable and debuggable. While far from a full-fledged accessibility solution, the prototype serves as a useful research vehicle for exploring gesture-based navigation and provides a foundation for future work on customization and accessibility-focused user studies.

Acknowledgements

I thank Prof. Xiaojun Bi for teaching this course and providing the conceptual tools that enabled me to design and run a within-subjects quantitative study. His project presentation and review session were particularly helpful in revealing weaknesses in my initial prototype and pushing me to refine the design, extend the scope of the project, and tighten the evaluation.

I am grateful to my friends Anusha Subash Gorawar, Shreyas B. S., and Samarakshni Meera for serving as Participants 1, 2, and 3 in the study, patiently drawing repeated gestures and giving concrete feedback on what felt natural or confusing in the interface.

I owe special thanks to my grandmother, Dr. Upma Trivedi, for participating as Participant 4 despite her motor impairment, and for helping me reason through how to fix the recognizer’s confusion between $<$ and $>$ as well as between $\wedge$ and $\vee$. Her support and perspective were central to the accessibility motivation of this project.

Finally, I acknowledge the use of ChatGPT as a writing assistant for refining the English in this report and for helping me structure and typeset the document in latex template format on overleaf. Any remaining errors are my own.

References

- [1] Jacob O. Wobbrock, Andrew D. Wilson, and Yang Li. Gestures without libraries, toolkits or training: A \$1 recognizer for user interface prototypes. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2007.
- [2] Opera Software. Mouse Gestures in Opera. Online documentation, accessed 2025.
- [3] Lifehacker. How to Use Mouse Gestures on Browsers. Available at: <https://lifehacker.com/tech/how-use-mouse-gestures-on-browsers>. Accessed 2025.